From Raw Ad Clicks to a Root-Cause Diagnosis

A plain-language guide to the data pipeline and dashboard —
for readers who don't write SQL, with every SQL script included for those who do.

AeroOps / AdOps Root-Cause Analyst · InMobi Click-a-thon 2026
Database: `ganesh` on ClickHouse Cloud

1. What This System Does (No Jargon)

Imagine a hospital heart-rate monitor. When it beeps, it doesn't tell you why the patient's heart rate changed — a nurse still has to run over, check the chart, and figure out what happened. This project builds the equivalent for advertising revenue: when the money coming in suddenly jumps or drops, the system doesn't just say "something changed." It automatically investigates *why*, and reports back in seconds with a plain-English explanation backed by real numbers.

Three jobs, done automatically instead of by a person clicking through dashboards for hours:

- **Notice** — spot the exact hour where revenue moved further from "normal" than random noise would explain.
- **Investigate** — automatically check every way of slicing the business (by country, by app, by advertiser, by device, and eight other angles) to find which slice is actually responsible.
- **Explain** — write a short, honest summary that names the responsible segment (or says "nothing dominates, this is broad-based") and states which other explanations were checked and ruled out.

> **Why this matters**
>
> At InMobi's scale, billions of ad requests flow through every day. A 1% revenue move is real money. Today, finding the cause means a person manually slicing dashboards for hours. This system turns that into a few seconds of automated, evidence-backed detective work.

2. The Ad Business, in Plain English

Every time an app wants to show an ad, it goes through a funnel. Picture it like a job interview process — lots of applicants at the start, fewer make it through each round:

Step	Plain-English meaning
Request	An app asks: "do you have an ad to show me right now?" This is the very top of the funnel — every single ask counts, whether or not it's answered.
Fill	An advertiser said yes and handed over an ad. Not every request gets filled — sometimes there's simply no matching ad available.
Impression	The ad actually got shown on someone's screen.
Click	The person tapped the ad.
Revenue	The money earned — collected once an impression happens.

Beyond the funnel, every ad request also has **attributes** — which app it happened in, which country the person was in, what phone they used, which advertiser paid for it, and so on. Think of it like a pizza: the same pizza can be sliced by topping, by size, or by who ordered it. Each way of slicing is called a **dimension**. This system checks **11 different dimensions** every time it investigates a revenue move, so it never misses an angle a human might forget to check.

3. Where the Data Comes From (Source Tables)

There are two kinds of tables here, and the difference matters:

- **A fact table** is like a cash register receipt log — one line per event, millions of lines, constantly growing. Here, that's `ad_events`: one row per ad request.
- **A dimension table** is like a customer directory or a product catalog — a short reference list you look things up in. Here, that's the app list, the advertiser list, and the device/geography list.

All four source tables live in the read-only `quvia_hack` database — this is the untouched data as it was handed over, ~9 million ad-request rows spanning 5 weeks.

3.1 The event log — one row per ad request

```
CREATE TABLE quvia_hack.ad_events
(
    `event_time` DateTime64(3),
    `app_id` LowCardinality(String),
    `geo_device_id` LowCardinality(String),
    `advertiser_id` LowCardinality(String),
    `ad_format` LowCardinality(String),
    `is_filled` UInt8,
    `is_impression` UInt8,
    `is_click` UInt8,
    `revenue` Float64
)
ENGINE = SharedMergeTree(...)
ORDER BY event_time
```

In plain terms: every row says *"at this exact moment, this app asked for an ad for this person on this device — here's who paid for it, what format it was, whether it got filled/shown/clicked, and how much money it made."* `advertiser_id` is blank when a request wasn't filled, because nobody paid for an ad that was never shown.

3.2 The three reference lists (dimension tables)

Who is paying for the ad, and how they're billed:

```
CREATE TABLE quvia_hack.advertisers
(
    `advertiser_id` String,
    `vertical` LowCardinality(String),      -- e.g. gaming, finance, travel
    `campaign_type` LowCardinality(String)  -- CPM, CPC, or CPI
)
ORDER BY advertiser_id
```

Where the ad is shown:

```
CREATE TABLE quvia_hack.apps
(
    `app_id` String,
    `category` LowCardinality(String),        -- e.g. gaming, news, finance
    `publisher_tier` LowCardinality(String)   -- tier_1 / tier_2 / tier_3 (quality tier)
)
ORDER BY app_id
```

Who is seeing the ad, and on what:

```
CREATE TABLE quvia_hack.geo_device
(
    `geo_device_id` String,
    `region` LowCardinality(String),        -- NAM, EU, APAC, LATAM, MEA
    `country` LowCardinality(String),       -- US, CA, UK, DE, IN, ...
    `device_model` LowCardinality(String),  -- iPhone 13, Galaxy A54, ...
    `os_version` LowCardinality(String)     -- iOS 17.2, Android 14, ...
)
ORDER BY geo_device_id
```

4. The Journey — From One Giant Table to a Dashboard You Can Click

Nobody can eyeball 9 million individual rows and spot a problem. The system builds several layers on top of the raw data, each one boiling the previous layer down further, until what's left is small enough to check instantly. Here is every layer, in order.

Step 1 — Phone books (dictionaries)

A **dictionary** in ClickHouse is exactly what it sounds like: a fast lookup table. Instead of re-reading the entire advertiser list every time, the system loads it once into memory as a phone book: "look up this advertiser_id, get back its vertical and campaign_type." There are three of these, one per reference list.

```
CREATE DICTIONARY ganesh.advertisers_dict
(
    `advertiser_id` String,
    `vertical` String,
    `campaign_type` String
)
PRIMARY KEY advertiser_id
SOURCE(CLICKHOUSE(TABLE 'advertisers' DB 'quvia_hack'))
LIFETIME(MIN 300 MAX 600)
LAYOUT(COMPLEX_KEY_HASHED())
```

(`apps_dict` and `geo_device_dict` are built the same way, pointed at `apps` and `geo_device` respectively. Full scripts are in Appendix A.)

Step 2 — Hourly rollups: collapsing 9 million rows into hourly totals

This is the biggest simplification in the whole pipeline: instead of keeping every single ad request, the system adds them all up into one number per hour (and, separately, one number per hour per dimension value — e.g. one number for "US, 6am on June 21"). Two tables hold these rollups:

- `metrics_1h` — one row per hour, no breakdown (840 rows total: 5 weeks x 168 hours/week).
- `dim_metrics_1h` — one row per hour *per dimension value*, across all 11 dimensions (e.g. a separate row for every country, every hour).

> **A technical wrinkle, explained simply**
>
> These tables don't store a plain number like "142 requests." They store a special half-finished "running subtotal" (ClickHouse calls the table type `AggregatingMergeTree`). Think of it like a cash register tape that hasn't been added up yet — you must run a totaling function (`countMerge()`, `sumMerge()`) to get the real number back out. This exists purely for speed: ClickHouse can update these running subtotals far faster than it could re-sum 9 million rows every single query. Every query in this guide that reads these two tables uses the correct `*Merge()` function for exactly this reason.

The rule that keeps metrics_1h filled in (a materialized view):

```
CREATE MATERIALIZED VIEW ganesh.mv_metrics_1h TO ganesh.metrics_1h
AS SELECT
    toStartOfHour(event_time) AS bucket,
    countState()            AS requests,
    sumState(is_filled)     AS fills,
    sumState(is_impression) AS impressions,
    sumState(is_click)      AS clicks,
    sumState(revenue)       AS revenue
FROM quvia_hack.ad_events
GROUP BY bucket
```

A **materialized view** is just a standing instruction: "whenever new rows land in `ad_events`, automatically compute this summary and file it away." There are 11 more of these, one per dimension (e.g. one that summarizes by `country`, one by `app_id`, etc.) — all feeding the single shared `dim_metrics_1h` table. Full list in Appendix A.

Step 3 — What does "normal" even mean? (the baseline)

You can't call a Saturday afternoon abnormal just because it looks different from a Tuesday morning — weekends are naturally quieter. So "normal" is defined *per time slot*: for every combination of day-of-week and hour-of-day (Monday 6am, Monday 7am, ... Sunday 11pm — 168 slots total), the system looks across all 5 weeks of history and asks two questions:

- **Median** — line up every historical value for this slot from smallest to largest and take the middle one. (Plain English: the median is just "the usual value," and unlike an average, one freak outlier day can't drag it around.)
- **MAD (Median Absolute Deviation)** — for that same slot, measure how far each historical value typically strays from the median, then take the median of *that*. This is "how much this slot normally wobbles." A slot that's always rock-steady gets a small MAD; a slot that's naturally noisy gets a bigger one.

```
CREATE MATERIALIZED VIEW ganesh.mv_baseline_1h_refresh
REFRESH EVERY 1 DAY OFFSET 1 HOUR TO ganesh.baseline_1h
AS WITH per_bucket AS (
    SELECT bucket, sumMerge(revenue) AS revenue_bucket
    FROM ganesh.metrics_1h GROUP BY bucket
),
med AS (SELECT median(revenue_bucket) AS m FROM per_bucket)
SELECT
    toDayOfWeek(bucket) AS dow, toHour(bucket) AS hod,
    median(revenue_bucket) AS median_revenue,
    median(abs(revenue_bucket - med.m)) AS mad_revenue
FROM per_bucket CROSS JOIN med
GROUP BY dow, hod
```

(`baseline_dim_1h` does the identical calculation, but separately for every dimension value — e.g. a separate "what's normal for country=US at Monday 6am" baseline. Full script in Appendix A.)

Step 4 — The alarm system: deciding what counts as "abnormal"

With a median and a MAD in hand for every slot, the system computes a **robust z-score** for every actual hour: roughly, *"how many typical wobbles away from normal is this?"* A z-score of 1 means "about as far off as a normal wobble." A z-score of 3 or more means the deviation is far bigger than the slot's usual noise — that's flagged as an anomaly.

> **Why not just use a flat percentage threshold?**

An earlier version of this system flagged anything more than 15% off as an anomaly. That sounds reasonable, but it falsely flagged **21.5% of all hours** — because some slots are naturally noisy and routinely swing by more than 15% for no reason at all, while other slots are so steady that even a 6% move is a genuine, statistically real anomaly. The z-score fixes this because it compares each hour to *its own slot's normal wobble*, not one fixed number for the whole business.

```
-- flagged if robust_z > 3
robust_z = abs(revenue_actual - revenue_median) / (1.4826 * mad_revenue)
```

(The constant 1.4826 is a standard statistics correction factor that makes MAD comparable to a normal standard deviation — you don't need to know why, just that it's a well-established fixed number, not something invented for this project.)

```
CREATE MATERIALIZED VIEW ganesh.anomaly_watch_1h
REFRESH EVERY 1 HOUR APPEND TO ganesh.anomalies_1h
AS WITH candidates AS (
    SELECT bucket, sumMerge(revenue) AS revenue
    FROM ganesh.metrics_1h
    WHERE bucket NOT IN (SELECT bucket FROM ganesh.anomalies_1h)   -- don't re-flag the same hour twice
    GROUP BY bucket
)
SELECT c.bucket, c.revenue, b.median_revenue AS expected_revenue,
       (c.revenue - b.median_revenue) / b.median_revenue AS pct_dev,
       abs(c.revenue - b.median_revenue) / (1.4826 * b.mad_revenue) AS robust_z,
       now() AS detected_at
FROM candidates c
INNER JOIN ganesh.baseline_1h b ON b.dow = toDayOfWeek(c.bucket) AND b.hod = toHour(c.bucket)
WHERE (abs(c.revenue - b.median_revenue) / (1.4826 * b.mad_revenue)) > 3
```

Step 5 — The detective's toolbox: drilling down into "why"

Once an hour is flagged, the system needs to find out *which slice* of the business caused it. For every one of the 11 dimensions, it compares "what actually happened in each value of that dimension" against "what was normal for that value at this exact time slot," and computes a **delta** (the gap) for each one.

The key trick: it ranks each dimension's values *against each other, within that same dimension* — never pooling all 11 dimensions' deltas into one shared pot. That gives a **percent of total delta** per dimension: e.g. "within the country dimension, the US alone explains 30% of the country-attributable gap." A high number for one specific value means that value is likely the culprit; a low top number across the board means the movement is broad-based and no single segment is responsible.

A bug this exact rule caught and fixed

An earlier version summed all 11 dimensions' deltas into one shared total before computing percentages. That deflated every percentage by roughly 10x and could point at the wrong "most responsible" segment. Fixing it meant computing the percentage *separately per dimension* — which is why the SQL below has `PARTITION BY dimension_name` in it. Removing that one clause silently breaks the diagnosis.

```
delta = revenue_now - revenue_expected                      -- per dimension value, same hour
pct_of_total_delta = delta / SUM(delta) OVER (PARTITION BY dimension_name)
```

Step 6 — The API: a waiter between the database and the screen

A small backend program (built with Python's FastAPI framework) sits between the dashboard and ClickHouse. Think of it as a waiter: the dashboard never talks to the database directly — it asks the waiter a question ("what happened at 6am on June 21st?"), the waiter runs the right SQL query, and brings back the answer as a tidy list of numbers. This keeps the heavy lifting entirely inside ClickHouse and keeps the screen simple and fast.

Step 7 — The dashboard: what a person actually sees

The dashboard is the only part of this whole pipeline a non-technical person ever looks at. Section 8 walks through exactly what's on screen and how the drill-down works.

5. Table Reference — What Every Table Is For

Name	Plain-English purpose
quvia_hack.ad_events	The raw firehose. One row per ad request. Read-only, untouched.
quvia_hack.advertisers / apps / geo_device	The three reference lists (who's paying, where ads show, who's watching).
ganesh.*_dict (3 dictionaries)	Fast phone-book lookups for the three reference lists.
ganesh.metrics_1h	Hourly totals for the whole business, no breakdown. 840 rows.
ganesh.dim_metrics_1h	Hourly totals broken out by all 11 dimensions.
ganesh.baseline_1h	"What's normal" (median + typical wobble) for each of the 168 day-of-week / hour-of-day slots.
ganesh.baseline_dim_1h	Same as above, but separately for every dimension value.
ganesh.anomalies_1h	The running list of flagged abnormal hours — the alarm log.
ganesh.anomaly_watch_1h	The standing rule that checks for new anomalies every hour and appends to anomalies_1h.

6. The Formulas, Explained Like You're Five

Term	Plain-English meaning	Formula
Fill rate	Out of everyone who asked for an ad, what share actually got one?	fills / requests
CTR	Out of everyone who saw an ad, what share tapped it?	clicks / impressions
eCPM	Money earned per 1,000 ad views — the standard "price tag" for ads.	revenue / impressions * 1000
Revenue per fill	Average money made per ad actually handed out.	revenue / fills
Median	The "usual" value — line up history, pick the middle one. Not fooled by one crazy day.	middle of sorted values
MAD	How much this time slot normally wobbles, day to day.	median(|value - median|)
pct_dev	How far off (as a percent) an hour is from what's normal for it.	(actual - median) / median
Robust z-score	How many "normal wobbles" away from usual this hour is. Above 3 = flagged.	|actual - median| / (1.4826 * MAD)
Delta	The plain dollar gap between what happened and what was expected.	revenue_now - revenue_expected
% of total delta	Within one dimension, what share of the total gap this one value explains.	delta / SUM(delta) per dimension

Term	Plain-English meaning	Formula
pct_change (factors)	How far a factor (volume/fill-rate/price) is from its usual value for this slot.	(now - typical) / typical

The revenue identity — the one equation that ties all of the above together. When revenue moves, this is the order to check factors in: did fewer people ask for ads (volume), did fewer of those asks get filled (fill rate), or did each filled ad simply earn less (price)?

```
Revenue  =  Requests  x  Fill rate  x  (Impressions / Fills)  x  eCPM / 1000
```

Since almost every fill becomes an impression, this simplifies in practice to:

```
Revenue  is approximately equal to  Requests  x  Fill rate  x  eCPM / 1000
```

7. The SQL Query Library, Explained

These are the exact queries the backend runs. Each one answers one specific question a human investigator would ask. Every number shown on the dashboard traces back to one of these.

Query 1 — "Is this hour actually abnormal?" (Detection)

Takes one hour, looks up what's normal for that exact day-of-week/hour-of-day slot, and returns the gap and the z-score.

```
WITH current AS (
    SELECT bucket, sumMerge(revenue) AS revenue
    FROM ganesh.metrics_1h
    WHERE bucket = {bucket:DateTime}
    GROUP BY bucket
)
SELECT
    c.bucket, c.revenue, b.median_revenue AS expected_revenue,
    (c.revenue - b.median_revenue) / b.median_revenue AS pct_dev,
    abs(c.revenue - b.median_revenue) / (1.4826 * b.mad_revenue) AS robust_z
FROM current c
INNER JOIN ganesh.baseline_1h b
    ON b.dow = toDayOfWeek(c.bucket) AND b.hod = toHour(c.bucket)
```

Query 2 — "Show me everything currently flagged" (the alarm log)

```
SELECT bucket, revenue, expected_revenue, pct_dev, robust_z
FROM ganesh.anomalies_1h
ORDER BY robust_z DESC
```

Query 3 — "Which segment is responsible?" (Contribution ranking, all 11 dimensions)

This is the core investigative query — the one the "contribution strip" on the dashboard is built from. It checks all 11 dimensions at once and ranks each one's top suspects.

```
WITH current_window AS (
    SELECT dimension_name, dimension_value, sumMerge(revenue) AS revenue_now
    FROM ganesh.dim_metrics_1h
    WHERE bucket = {bucket:DateTime}
    GROUP BY dimension_name, dimension_value
),
joined AS (
    SELECT c.dimension_name, c.dimension_value, c.revenue_now,
           b.median_revenue AS revenue_expected,
           (c.revenue_now - b.median_revenue) AS delta
    FROM current_window c
    INNER JOIN ganesh.baseline_dim_1h b
        ON b.dimension_name = c.dimension_name AND b.dimension_value = c.dimension_value
        AND b.dow = toDayOfWeek({bucket:DateTime}) AND b.hod = toHour({bucket:DateTime})
)
SELECT dimension_name, dimension_value, delta,
       delta / sum(delta) OVER (PARTITION BY dimension_name) AS pct_of_total_delta,
       row_number() OVER (PARTITION BY dimension_name ORDER BY abs(delta) DESC) AS rnk
FROM joined
QUALIFY rnk <= 5
ORDER BY dimension_name, rnk
```

Query 4 — "Zoom into one dimension" (e.g. top countries, top apps)

Same idea as Query 3, but also brings back fill rate and revenue-per-fill for each top segment, so you can immediately see whether that segment's problem is volume, fill rate, or price. This is what powers the drill-down panel when you click a dimension on the dashboard.

```
WITH current_window AS (
    SELECT dimension_name, dimension_value,
           countMerge(requests) AS requests_now, sumMerge(fills) AS fills_now,
           sumMerge(impressions) AS impressions_now, sumMerge(revenue) AS revenue_now
    FROM ganesh.dim_metrics_1h
    WHERE bucket = {bucket:DateTime}
      AND dimension_name IN ('country', 'app_id', 'advertiser_id')   -- or any of the 11
    GROUP BY dimension_name, dimension_value
),
joined AS (
    SELECT c.dimension_name, c.dimension_value, c.requests_now, c.fills_now,
           c.impressions_now, c.revenue_now, b.median_revenue AS revenue_expected,
           (c.revenue_now - b.median_revenue) AS delta,
           c.fills_now / nullIf(c.requests_now, 0) AS fill_rate_now,
           c.revenue_now / nullIf(c.fills_now, 0) AS revenue_per_fill_now
    FROM current_window c
    INNER JOIN ganesh.baseline_dim_1h b
        ON b.dimension_name = c.dimension_name AND b.dimension_value = c.dimension_value
        AND b.dow = toDayOfWeek({bucket:DateTime}) AND b.hod = toHour({bucket:DateTime})
)
SELECT dimension_name, dimension_value, revenue_now, revenue_expected, delta,
       delta / sum(delta) OVER (PARTITION BY dimension_name) AS pct_of_total_delta,
       fill_rate_now, revenue_per_fill_now,
       row_number() OVER (PARTITION BY dimension_name ORDER BY abs(delta) DESC) AS rnk
FROM joined
QUALIFY rnk <= 5
ORDER BY dimension_name, rnk
```

Query 5 — "Was it volume, fill rate, or price?" (Factor decomposition)

```
WITH per_bucket AS (
    SELECT bucket, countMerge(requests) AS requests, sumMerge(fills) AS fills, sumMerge(revenue) AS revenue
    FROM ganesh.metrics_1h
    WHERE toDayOfWeek(bucket) = toDayOfWeek({bucket:DateTime})
      AND toHour(bucket) = toHour({bucket:DateTime})
    GROUP BY bucket
),
stats AS (
    SELECT median(requests) AS requests_typical,
           median(fills / requests) AS fill_rate_typical,
           median(revenue / fills) AS revenue_per_fill_typical
    FROM per_bucket
)
SELECT
    p.bucket, p.requests, s.requests_typical,
    (p.requests - s.requests_typical) / s.requests_typical AS requests_pct_change,
    p.fills / p.requests AS fill_rate_now, s.fill_rate_typical,
    (p.fills/p.requests - s.fill_rate_typical) / s.fill_rate_typical AS fill_rate_pct_change,
    p.revenue / p.fills AS revenue_per_fill_now, s.revenue_per_fill_typical,
    (p.revenue/p.fills - s.revenue_per_fill_typical) / s.revenue_per_fill_typical AS revenue_per_fill_pct_change
FROM per_bucket p, stats s
WHERE p.bucket = {bucket:DateTime}
```

Query 6 — "Is this just a normal weekend?" (Weekday vs weekend check)

```
SELECT dow, hod, median_revenue, mad_revenue
FROM ganesh.baseline_1h
WHERE hod = 12   -- or any hour, to compare all 7 days side by side
ORDER BY dow
```

dow (day-of-week) 1 = Monday ... 7 = Sunday. Comparing all 7 rows at the same hour shows at a glance whether weekends are naturally lower — which is exactly why the baseline is built per day-of-week instead of one flat average for the whole business.

Query 7 — "Show me the whole day" (Full-day trend)

```
WITH day AS (
    SELECT bucket, sumMerge(revenue) AS revenue
    FROM ganesh.metrics_1h
    WHERE toDate(bucket) = {date:Date}
    GROUP BY bucket
)
SELECT d.bucket, d.revenue, b.median_revenue AS expected
FROM day d
INNER JOIN ganesh.baseline_1h b ON b.dow = toDayOfWeek(d.bucket) AND b.hod = toHour(d.bucket)
ORDER BY d.bucket
```

This is how a single flagged hour was discovered to actually be part of an all-day, sustained revenue collapse rather than a one-hour blip — plotting the whole day next to its expected values makes that instantly visible.

8. The Dashboard Walkthrough — What You See and Click

Everything on screen is fed by the queries in Section 7 — nothing is invented client-side beyond simple formatting (rounding, adding a $ sign). Here is the screen, top to bottom:

On screen	What it shows	Fed by
Sidebar — flagged anomalies	Every currently-flagged hour, ranked by z-score, color-coded, with a weekday/weekend badge.	Query 2
KPI strip	Revenue, expected revenue, % deviation, z-score for the selected hour, at a glance.	Query 1
Diagnosis card	A one-paragraph plain-English summary — states the day of week (so weekend seasonality is visibly ruled out), the dominant factor, and the responsible segment (or "broad-based, nothing dominates").	Queries 1, 3, 4, 5 — narrated by rule, not by a language model
Full-day trend chart	Actual vs. expected revenue for all 24 hours of that day — shows whether this was a one-hour blip or an all-day event.	Query 7
Factor tiles	Request volume / fill rate / price, each with its % change vs. typical, and which one is flagged as the primary driver.	Query 5
Contribution strip	All 11 dimensions, ranked by how concentrated their top segment is — click any row to open its drill-down.	Query 3
Drill-down panel	Bar chart + table of the top segments within whichever dimension you clicked, including fill rate and revenue-per-fill for each.	Query 4

Clicking through: a viewer picks a flagged hour from the sidebar → reads the diagnosis and KPI strip immediately → checks the trend chart to see if it's a blip or a sustained event → scans the contribution strip to see which dimension is most concentrated → clicks that dimension to see exactly which value (which country, which app, ...) is responsible, with the underlying fill-rate and price numbers right there to confirm the story.

9. A Real Bug We Found (and How We Caught It)

This system's whole promise is "every number is real and traceable — nothing is made up." That promise is only worth something if it's actually checked. During review, a reconciliation check — adding up the revenue for every value inside one dimension and comparing it to the known company-wide total — turned up a real problem:

The vertical dimension was reporting exactly double

Every one of the 11 dimensions summed back to the correct total revenue ($17,020.36) *except* "vertical" (gaming, finance, travel, etc.), which summed to exactly $34,040.73 — precisely 2x. The cause: the one-time backfill step for that dimension had accidentally been run twice, and because of how the hourly rollups store "running subtotals" (see Step 2 in Section 4), the two runs quietly added together instead of overwriting each other. Any diagnosis that had pointed to a vertical as the culprit would have been reporting numbers twice as large as reality.

The fix: delete the doubled subtotal, rebuild it once from scratch, then rebuild the "what's normal for vertical" baseline from the corrected numbers. Verified afterward by re-running the same reconciliation check.

```
-- 1. Remove the doubled 'vertical' aggregate states
ALTER TABLE ganesh.dim_metrics_1h DELETE WHERE dimension_name = 'vertical';

-- 2. Re-run the backfill exactly once
INSERT INTO ganesh.dim_metrics_1h
SELECT
    toStartOfHour(event_time) AS bucket,
    'vertical' AS dimension_name,
    dictGet('ganesh.advertisers_dict', 'vertical', advertiser_id) AS dimension_value,
    countState() AS requests, sumState(is_filled) AS fills,
    sumState(is_impression) AS impressions, sumState(is_click) AS clicks,
    sumState(revenue) AS revenue
FROM quvia_hack.ad_events
GROUP BY bucket, dimension_value;

-- 3 & 4. Delete and recompute the now-stale 'vertical' baseline the same way
```

This is included here deliberately: it's the clearest illustration of why every claim in this system is checked against a re-derivable number, rather than trusted at face value.

10. Glossary

Term	Plain-English meaning
Aggregate / rollup	Adding many small numbers into one bigger summary number (e.g. millions of events into one hourly total).
AggregatingMergeTree	A ClickHouse table type that stores half-finished running subtotals instead of final numbers, for speed. Must be read with a *Merge() function.
API / backend	The small program that sits between the dashboard and the database, fetching answers on request.
Baseline	The "normal" value for a given time slot, used as the yardstick to compare an actual hour against.
Bucket	One hour of time, e.g. "2026-06-21 06:00" — the basic unit everything is measured in.
CTR (click-through rate)	The share of people who saw an ad and tapped it: clicks / impressions.
Delta	The plain dollar gap between what actually happened and what was expected.
Dictionary (ClickHouse)	A fast in-memory lookup table, like a phone book, built from a reference table.
Dimension	A way of slicing the data — by country, by app, by advertiser, etc. There are 11 in this system.
Dimension table	A reference list (who's advertising, which apps exist) — short and slow-changing, unlike the fact table.
Dow / hod	Shorthand for "day of week" and "hour of day" — the two numbers used to define a recurring time slot.
eCPM	Effective revenue per 1,000 ad views — the standard price benchmark for ads.
Fact table	The big, ever-growing event log — one row per real-world event (here, ad_events).
Fill / fill rate	A fill is a request that got an ad back. Fill rate is the share of requests that got filled.
Impression	An ad that was actually shown on screen (as opposed to just handed over).
Localized (vs. broad-based)	A movement is "localized" when one specific segment (e.g. one country) explains most of it; "broad-based" means no single segment dominates.
MAD (Median Absolute Deviation)	How much a time slot's value normally wobbles day to day — the "typical noise level" for that slot.
Materialized view	A standing rule in ClickHouse that automatically recomputes a summary whenever new source data arrives.
Median	The middle value when you line every historical value up in order — the "usual" value, immune to one freak outlier.
Pct_dev	Percent deviation — how far off (as a percentage) an actual value is from its expected/median value.
Pct_of_total_delta	Within one dimension, what share of the total gap one specific value (e.g. one country) is responsible for.

Term	Plain-English meaning
Request	An app asking for an ad to show — the very top of the funnel.
Revenue per fill	Average money earned per ad actually handed out: revenue / fills.
Robust z-score	How many "typical wobbles" away from normal a value is. Above 3 is flagged as a real anomaly, not noise.
Ruled out	A possible cause that was checked and found to be normal — explicitly reported, not just silently ignored.
Seasonality	Predictable, repeating patterns (weekends are quieter, certain hours are busier) that should never by themselves count as an anomaly.
Vertical	The industry an advertiser belongs to — gaming, finance, travel, etc.

Appendix A — Full SQL: Tables, Dictionaries & Materialized Views

Verbatim CREATE statements as they exist in the ganesh database. Grouped in the order data flows through them.

A.1 Dictionaries

```
CREATE DICTIONARY ganesh.advertisers_dict
(
    `advertiser_id` String,
    `vertical` String,
    `campaign_type` String
)
PRIMARY KEY advertiser_id
SOURCE(CLICKHOUSE(TABLE 'advertisers' DB 'quvia_hack'))
LIFETIME(MIN 300 MAX 600)
LAYOUT(COMPLEX_KEY_HASHED())
```

```
CREATE DICTIONARY ganesh.apps_dict
(
    `app_id` String,
    `category` String,
    `publisher_tier` String
)
PRIMARY KEY app_id
SOURCE(CLICKHOUSE(TABLE 'apps' DB 'quvia_hack'))
LIFETIME(MIN 300 MAX 600)
LAYOUT(COMPLEX_KEY_HASHED())
```

```
CREATE DICTIONARY ganesh.geo_device_dict
(
    `geo_device_id` String,
    `region` String,
    `country` String,
    `device_model` String,
    `os_version` String
)
PRIMARY KEY geo_device_id
SOURCE(CLICKHOUSE(TABLE 'geo_device' DB 'quvia_hack'))
LIFETIME(MIN 300 MAX 600)
LAYOUT(COMPLEX_KEY_HASHED())
```

A.2 Hourly rollup tables

```
CREATE TABLE ganesh.metrics_1h
(
    `bucket` DateTime,
    `requests` AggregateFunction(count),
    `fills` AggregateFunction(sum, UInt8),
    `impressions` AggregateFunction(sum, UInt8),
    `clicks` AggregateFunction(sum, UInt8),
    `revenue` AggregateFunction(sum, Float64)
)
ENGINE = AggregatingMergeTree
ORDER BY bucket
```

```
CREATE TABLE ganesh.dim_metrics_1h
(
    `bucket` DateTime,
    `dimension_name` LowCardinality(String),
    `dimension_value` LowCardinality(String),
    `requests` AggregateFunction(count),
    `fills` AggregateFunction(sum, UInt8),
    `impressions` AggregateFunction(sum, UInt8),
    `clicks` AggregateFunction(sum, UInt8),
    `revenue` AggregateFunction(sum, Float64)
)
ENGINE = AggregatingMergeTree
ORDER BY (dimension_name, dimension_value, bucket)
```

A.3 The 12 materialized views that keep the rollups fed

```
CREATE MATERIALIZED VIEW ganesh.mv_metrics_1h TO ganesh.metrics_1h AS
SELECT toStartOfHour(event_time) AS bucket,
       countState() AS requests, sumState(is_filled) AS fills,
       sumState(is_impression) AS impressions, sumState(is_click) AS clicks,
       sumState(revenue) AS revenue
FROM quvia_hack.ad_events GROUP BY bucket
```

```
-- Pattern repeated for all 11 dimensions (ad_format shown; swap the dimension_value expression
-- and 'dimension_name' literal for the other 10: app_id, campaign_type, category, country,
-- device_model, os_version, publisher_tier, region, vertical, advertiser_id)
CREATE MATERIALIZED VIEW ganesh.mv_dim_ad_format_1h TO ganesh.dim_metrics_1h AS
SELECT toStartOfHour(event_time) AS bucket,
       'ad_format' AS dimension_name, ad_format AS dimension_value,
       countState() AS requests, sumState(is_filled) AS fills,
       sumState(is_impression) AS impressions, sumState(is_click) AS clicks,
       sumState(revenue) AS revenue
FROM quvia_hack.ad_events GROUP BY bucket, dimension_value

-- Dimensions looked up via a dictionary instead of a raw column, e.g. country:
CREATE MATERIALIZED VIEW ganesh.mv_dim_country_1h TO ganesh.dim_metrics_1h AS
SELECT toStartOfHour(event_time) AS bucket,
       'country' AS dimension_name,
       dictGet('ganesh.geo_device_dict', 'country', geo_device_id) AS dimension_value,
       countState() AS requests, sumState(is_filled) AS fills,
       sumState(is_impression) AS impressions, sumState(is_click) AS clicks,
       sumState(revenue) AS revenue
FROM quvia_hack.ad_events GROUP BY bucket, dimension_value
```

A.4 Baseline tables and their refresh rules

```
CREATE TABLE ganesh.baseline_1h
(
    `dow` Int32, `hod` Int32,
    `median_revenue` Float64, `mad_revenue` Float64
)
ENGINE = MergeTree ORDER BY (dow, hod)
```

```
CREATE TABLE ganesh.baseline_dim_1h
(
    `dimension_name` String, `dimension_value` String,
    `dow` Int32, `hod` Int32,
    `median_revenue` Float64, `mad_revenue` Float64
)
ENGINE = MergeTree ORDER BY (dimension_name, dimension_value, dow, hod)
```

```
CREATE MATERIALIZED VIEW ganesh.mv_baseline_dim_1h_refresh
REFRESH EVERY 1 DAY OFFSET 1 HOUR TO ganesh.baseline_dim_1h
AS WITH per_bucket AS (
    SELECT dimension_name, dimension_value, bucket, sumMerge(revenue) AS revenue_bucket
    FROM ganesh.dim_metrics_1h
    GROUP BY dimension_name, dimension_value, bucket
),
medians AS (
    SELECT dimension_name, dimension_value, median(revenue_bucket) AS med_revenue
    FROM per_bucket GROUP BY dimension_name, dimension_value
)
SELECT p.dimension_name, p.dimension_value,
       toDayOfWeek(p.bucket) AS dow, toHour(p.bucket) AS hod,
       median(p.revenue_bucket) AS median_revenue,
       median(abs(p.revenue_bucket - m.med_revenue)) AS mad_revenue
FROM per_bucket p
INNER JOIN medians m USING (dimension_name, dimension_value)
GROUP BY p.dimension_name, p.dimension_value, dow, hod
```

A.5 The anomaly log and its watch rule

```
CREATE TABLE ganesh.anomalies_1h
(
    `bucket` DateTime, `revenue` Float64, `expected_revenue` Float64,
    `pct_dev` Float64, `robust_z` Float64, `detected_at` DateTime
)
ENGINE = MergeTree ORDER BY bucket
```

```
CREATE MATERIALIZED VIEW ganesh.anomaly_watch_1h
REFRESH EVERY 1 HOUR APPEND TO ganesh.anomalies_1h
AS WITH candidates AS (
    SELECT bucket, sumMerge(revenue) AS revenue
    FROM ganesh.metrics_1h
    WHERE bucket NOT IN (SELECT bucket FROM ganesh.anomalies_1h)
    GROUP BY bucket
)
SELECT c.bucket, c.revenue, b.median_revenue AS expected_revenue,
       (c.revenue - b.median_revenue) / b.median_revenue AS pct_dev,
       abs(c.revenue - b.median_revenue) / (1.4826 * b.mad_revenue) AS robust_z,
       now() AS detected_at
FROM candidates c
INNER JOIN ganesh.baseline_1h b ON b.dow = toDayOfWeek(c.bucket) AND b.hod = toHour(c.bucket)
WHERE (abs(c.revenue - b.median_revenue) / (1.4826 * b.mad_revenue)) > 3
```

Appendix B — Full SQL: Query Library (clean copies)

Same nine queries as Section 7, without the surrounding explanation — for copy-paste use.

B.1 Detection

```
WITH current AS (
    SELECT bucket, sumMerge(revenue) AS revenue
    FROM ganesh.metrics_1h WHERE bucket = {bucket:DateTime} GROUP BY bucket
)
SELECT c.bucket, c.revenue, b.median_revenue AS expected_revenue,
       (c.revenue - b.median_revenue) / b.median_revenue AS pct_dev,
       abs(c.revenue - b.median_revenue) / (1.4826 * b.mad_revenue) AS robust_z
FROM current c
INNER JOIN ganesh.baseline_1h b ON b.dow = toDayOfWeek(c.bucket) AND b.hod = toHour(c.bucket)
```

B.2 Live anomaly list

```
SELECT bucket, revenue, expected_revenue, pct_dev, robust_z
FROM ganesh.anomalies_1h ORDER BY robust_z DESC
```

B.3 Contribution ranking (all dimensions)

```
WITH current_window AS (
    SELECT dimension_name, dimension_value, sumMerge(revenue) AS revenue_now
    FROM ganesh.dim_metrics_1h WHERE bucket = {bucket:DateTime}
    GROUP BY dimension_name, dimension_value
),
joined AS (
    SELECT c.dimension_name, c.dimension_value, c.revenue_now,
           b.median_revenue AS revenue_expected, (c.revenue_now - b.median_revenue) AS delta
    FROM current_window c
    INNER JOIN ganesh.baseline_dim_1h b
        ON b.dimension_name = c.dimension_name AND b.dimension_value = c.dimension_value
        AND b.dow = toDayOfWeek({bucket:DateTime}) AND b.hod = toHour({bucket:DateTime})
)
SELECT dimension_name, dimension_value, delta,
       delta / sum(delta) OVER (PARTITION BY dimension_name) AS pct_of_total_delta,
       row_number() OVER (PARTITION BY dimension_name ORDER BY abs(delta) DESC) AS rnk
FROM joined QUALIFY rnk <= 5 ORDER BY dimension_name, rnk
```

B.4 Dimension drill-down (with fill rate & price)

```
WITH current_window AS (
    SELECT dimension_name, dimension_value,
           countMerge(requests) AS requests_now, sumMerge(fills) AS fills_now,
           sumMerge(impressions) AS impressions_now, sumMerge(revenue) AS revenue_now
    FROM ganesh.dim_metrics_1h
    WHERE bucket = {bucket:DateTime} AND dimension_name IN ('country','app_id','advertiser_id')
    GROUP BY dimension_name, dimension_value
),
joined AS (
    SELECT c.dimension_name, c.dimension_value, c.requests_now, c.fills_now,
           c.impressions_now, c.revenue_now, b.median_revenue AS revenue_expected,
           (c.revenue_now - b.median_revenue) AS delta,
           c.fills_now / nullIf(c.requests_now, 0) AS fill_rate_now,
           c.revenue_now / nullIf(c.fills_now, 0) AS revenue_per_fill_now
    FROM current_window c
    INNER JOIN ganesh.baseline_dim_1h b
        ON b.dimension_name = c.dimension_name AND b.dimension_value = c.dimension_value
        AND b.dow = toDayOfWeek({bucket:DateTime}) AND b.hod = toHour({bucket:DateTime})
)
SELECT dimension_name, dimension_value, revenue_now, revenue_expected, delta,
       delta / sum(delta) OVER (PARTITION BY dimension_name) AS pct_of_total_delta,
       fill_rate_now, revenue_per_fill_now,
       row_number() OVER (PARTITION BY dimension_name ORDER BY abs(delta) DESC) AS rnk
FROM joined QUALIFY rnk <= 5 ORDER BY dimension_name, rnk
```

B.5 Factor decomposition

```
WITH per_bucket AS (
    SELECT bucket, countMerge(requests) AS requests, sumMerge(fills) AS fills, sumMerge(revenue) AS revenue
    FROM ganesh.metrics_1h
    WHERE toDayOfWeek(bucket) = toDayOfWeek({bucket:DateTime}) AND toHour(bucket) = toHour({bucket:DateTime})
    GROUP BY bucket
),
stats AS (
    SELECT median(requests) AS requests_typical,
           median(fills / requests) AS fill_rate_typical,
           median(revenue / fills) AS revenue_per_fill_typical
    FROM per_bucket
)
SELECT p.bucket, p.requests, s.requests_typical,
       (p.requests - s.requests_typical) / s.requests_typical AS requests_pct_change,
       p.fills / p.requests AS fill_rate_now, s.fill_rate_typical,
       (p.fills/p.requests - s.fill_rate_typical) / s.fill_rate_typical AS fill_rate_pct_change,
       p.revenue / p.fills AS revenue_per_fill_now, s.revenue_per_fill_typical,
       (p.revenue/p.fills - s.revenue_per_fill_typical) / s.revenue_per_fill_typical AS revenue_per_fill_pct_change
FROM per_bucket p, stats s WHERE p.bucket = {bucket:DateTime}
```

B.6 Weekday vs weekend check

```
SELECT dow, hod, median_revenue, mad_revenue
FROM ganesh.baseline_1h WHERE hod = 12 ORDER BY dow
```

B.7 Full-day trend

```
WITH day AS (
    SELECT bucket, sumMerge(revenue) AS revenue
    FROM ganesh.metrics_1h WHERE toDate(bucket) = {date:Date} GROUP BY bucket
)
SELECT d.bucket, d.revenue, b.median_revenue AS expected
FROM day d
INNER JOIN ganesh.baseline_1h b ON b.dow = toDayOfWeek(d.bucket) AND b.hod = toHour(d.bucket)
ORDER BY d.bucket
```